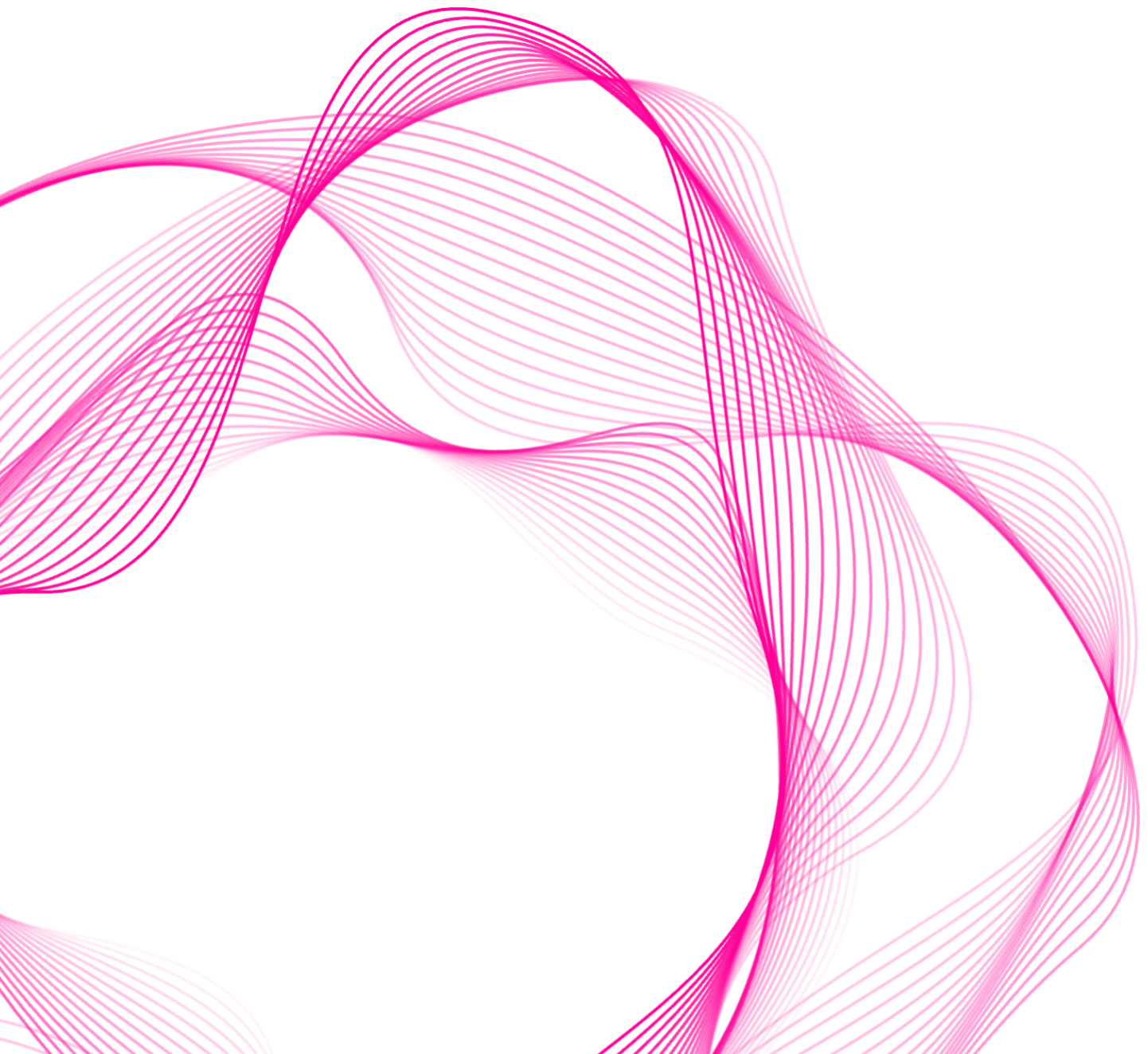


Prueba técnica agentes IA

Especificación de prueba técnica



Información del documento:

Título del documento	Prueba técnica desarrollo de deepagents
Nivel de seguridad	Interno
Propietario del documento	Jorge Capel

Registro de versiones		
Descripción	Versión	Fecha
Versión inicial del documento.	V1	14/03/2026

Acción	Autor	Fecha
Realiza	Jorge Capel	14/03/2026

DERECHOS DE USO:

La presente documentación es propiedad de NUNSYS GROUP y tiene carácter confidencial. Por tanto, no podrá ser objeto de reproducción total o parcial, tratamiento informático ni transmisión de ninguna forma o por cualquier medio, ya sea electrónico, telemático, mecánico, por fotocopia, registro o cualquier otro. Así mismo tampoco podrá ser objeto de préstamo, alquiler o cualquier forma de cesión de uso sin el permiso previo y por escrito de NUNSYS GROUP., titular del copyright. El incumplimiento de las limitaciones señaladas por cualquier persona que tenga acceso a la documentación será perseguido conforme a ley.

1. INTRODUCCIÓN	4
2. ALCANCE FUNCIONAL ESPERADO.....	4
2.1. CONSULTAS SOBRE ERP Y PRODUCCIÓN	4
2.2. CONSULTAS SOBRE DOCUMENTOS (RAG).....	5
2.3. API DE CONSULTA.....	5
2.4. EXPLICABILIDAD Y TRAZABILIDAD (MUY IMPORTANTE).....	6
2.5. MEMORIA CONVERSACIONAL (OPCIONAL)	6
3. REQUISITOS TÉCNICOS.....	6
3.1. FRAMEWORK Y ARQUITECTURA	6
4. ENTREGABLES.....	6
5. CRITERIOS DE EVALUACIÓN	7
5.1. FUNCIONALIDAD	7
5.2. CALIDAD DEL CÓDIGO.....	7
5.3. USO DE LANGCHAIN	7
5.4. ARQUITECTURA (CLAVE PARA PERFIL SENIOR)	7

1. Introducción

Esta Prueba de Concepto (POC) tiene como objetivo evaluar el diseño e implementación de un sistema multi-agente inteligente capaz de responder preguntas de negocio en lenguaje natural mediante la orquestación de múltiples fuentes de datos empresariales.

El sistema deberá integrar información procedente de un ERP (basado en la base de datos de Northwind) y de un sistema de producción accesible mediante API REST, siendo capaz de interpretar consultas, planificar acciones, ejecutar llamadas a diferentes fuentes y generar respuestas consolidadas, explicables y trazables.

Adicionalmente, la solución deberá incorporar capacidades de consulta sobre documentos mediante técnicas de recuperación aumentada (RAG), integrando almacenamiento vectorial y memoria conversacional.

A diferencia de una prueba técnica tradicional, esta POC está orientada a evaluar competencias propias de un perfil senior, incluyendo:

- Diseño de arquitecturas modulares y escalables
- Implementación de sistemas basados en agentes
- Integración de múltiples fuentes de datos (estructurados y no estructurados)
- Capacidad de razonamiento, planificación y trazabilidad
- Calidad del código y buenas prácticas de desarrollo

El desarrollo deberá entregarse en un **repositorio de Git accesible (GitHub, GitLab o similar)**, que será parte fundamental de la evaluación.

2. Alcance Funcional Esperado

El sistema deberá ser capaz de responder preguntas de negocio en lenguaje natural mediante la orquestación de múltiples fuentes de datos (ERP, sistema de producción y documentos).

2.1.Consultas sobre ERP y Producción

El sistema debe permitir resolver, al menos, los siguientes tipos de preguntas:

- “¿Qué pedidos pendientes tiene el cliente X y en qué estado de producción están?”
- “¿Qué pedidos están bloqueados y cuál es el motivo?”
- “¿Qué clientes tienen pedidos retrasados por problemas de producción?”

- “Dame un resumen del estado de los pedidos de este mes”

Para ello, el sistema deberá:

- Consultar datos del ERP (clientes, pedidos, importes)
- Consultar el sistema de producción vía API
- Combinar la información de ambas fuentes
- Generar una respuesta consolidada y comprensible

2.2.Consultas sobre documentos (RAG)

El sistema debe permitir:

- Subir documentos (PDF)
- Indexarlos en una base de datos vectorial
- Realizar preguntas sobre su contenido

Ejemplos:

- “¿Qué dice este documento sobre plazos de entrega?”
- “Resume los puntos clave del contrato”
- “¿Hay alguna penalización por retrasos?”

2.3.API de consulta

El sistema debe exponer al menos un endpoint:

- POST /api/query

Capacidades:

- Recibir una pregunta en lenguaje natural
- Procesarla mediante el sistema multi-agente
- Devolver una respuesta estructurada

Formato de respuesta esperado (ejemplo):

```
{
  "answer": "El cliente ALFKI tiene 3 pedidos pendientes...",
  "sources": ["ERP", "Producción"],
  "reasoning": [
    "Consulta ERP para pedidos",
    "Consulta API de producción",
    "Fusión de resultados"
  ]
}
```

2.4. Explicabilidad y trazabilidad (muy importante)

Cada respuesta debe incluir:

- Qué fuentes se han consultado
- Qué pasos se han ejecutado
- Qué lógica se ha seguido (aunque sea simplificada)

Esto es obligatorio para evaluar correctamente el razonamiento del sistema.

2.5. Memoria conversacional (Opcional)

El sistema debe opcionalmente:

- Recordar el contexto de la conversación
- Tener en cuenta al menos las últimas 5 interacciones
- Permitir preguntas encadenadas como:
 - “¿Y cuáles de esos pedidos están bloqueados?”
 - “¿Cuál es el impacto económico de esos?”

3. Requisitos Técnicos

La aplicación deberá cumplir con los siguientes requisitos:

3.1. Framework y arquitectura

- Uso de [deepAgentes de LangChain](#) como framework principal
- Diseño basado en agentes (multi-agent o agentic workflow)
- Una interfaz gráfica en chainlit o similares para poder realizar preguntas en lenguaje natural

4. Entregables

El candidato deberá proporcionar:

1. Código fuente completo de la aplicación.
2. Instrucciones claras sobre cómo configurar y ejecutar la aplicación.
3. Un archivo README.md que describa:
 - Arquitectura de la solución
 - Decisiones técnicas adoptadas

- Dependencias
 - Ejemplos de uso
4. Dockerfile y archivo docker-compose.yml para contenerizar y levantar la aplicación.
 5. (Opcional pero muy valorable) Un breve vídeo o demo mostrando la aplicación en funcionamiento.

5. Criterios de Evaluación

5.1. Funcionalidad

- Cumplimiento de todos los requisitos especificados
- Correcta integración de ERP, producción y documentos

5.2. Calidad del código

- Código limpio, estructurado y mantenible
- Buen uso de abstracciones
- Separación de responsabilidades

5.3. Uso de LangChain

- Correcta integración del modelo de lenguaje
- Uso adecuado de:
 - agentes
 - memoria
 - herramientas (tools)
- Diseño coherente del flujo de ejecución

5.4. Arquitectura (clave para perfil senior)

- Escalabilidad
- Extensibilidad
- Trazabilidad del sistema (logs, pasos, decisiones)